GutWell — Master Brief & Developer Handoff

Contents

GutWell — Master Brief & Developer Handoff

Audience: the new developer taking over GutWell (beginner-friendly — no prior knowledge of this codebase assumed). **Goal of this document:** explain what GutWell is, how it feels as a product (feature feedback), how the code is organized, and **exactly what to do next** to finish it.

- **App:** GutWell — a gut-health tracking iOS app (log food + symptoms -> discover your trigger foods)
- **Tech:** Expo SDK 54 · React Native · TypeScript · Supabase (database + login + server functions)
- **Owner:** Hojir / Parallel Labs Pte. Ltd. · **Bundle id:** `com.parallellabs.gutwell`
- **Repo:** https://github.com/jajrjavanmardi-debug/gutwell-app · branch `main`
- **Date:** 2 June 2026

How to read this document

1. **Part A — Feature & logic feedback:** how the app works as a product, what's smart, what feels off. Read this first to understand *why* some tasks matter.
2. **Part B — The code:** how to set it up, how it's organized, and the safe way to make changes.
3. **Part C — What still needs to be done:** a prioritized to-do list, from easy "first tasks" to bigger ones.
4. **Part D — How to proceed:** a suggested order of work for your first two weeks.
5. **Appendix:** what was already done, the full feature list, and reference links.

> **The single most important fact:** the app's backend (Supabase) is currently **paused**, so the app can't log in or load data until **Hojir restores it**. You can still set up and read the code, but to *run* the app fully, the backend must be active. See Part C -> "Owner-only tasks."

Part A — Feature & logic feedback

Written from the perspective of a real user with gut problems (bloating) who wants to find their trigger foods. This tells you which features are strong and which design choices are questionable — useful context before you change anything.

What's genuinely good (don't break these)

- **The core idea is exactly the user's problem:** "log food + symptoms -> see what triggers you." Everything should serve this loop.
- **Bristol stool chart** in the daily check-in — this is a real clinical tool and makes the app feel trustworthy and serious.
- **AI meal-photo logging** — snap a photo of a meal instead of typing it. This is the standout feature; it removes the friction that makes people quit food trackers.
- **One gut score + a streak** — a single daily number and a streak give the user an at-a-glance "am I improving?" and a reason to come back.
- **Data ownership** (export + delete your data) — important for health data and trust.

Design choices that are questionable (improvement opportunities)

1. ~~**The score dropped when you logged symptoms**~~ — [DONE] **already fixed (PR #26).** Honest logging is now credited, not punished.
2. ~~**The core insight (correlations) was 100% behind the paywall**~~ — [DONE] **already fixed (PR #26):** free users now see their single strongest trigger food before paying.
3. **The "Analysing…" screen in onboarding is fake** — it's a ~2.6-second animation that pretends to compute something. For a health app this hurts trust. *Suggestion: remove it, or make it show real setup progress.*
4. **The 7-question onboarding quiz -> "gut profile type" is mostly decorative** — it labels you (e.g. "Reactive Gut") but the answers don't really change how the app behaves. *Suggestion: either use the answers to personalize the experience, or simplify the quiz.*
5. **The app asks for your location to "find healthy food nearby"** — this is unrelated to the core loop and adds a privacy prompt. *Suggestion: cut it or make it clearly optional and late.*
6. **XP / levels / achievements feel juvenile for a health concern** — streaks fit, but "leveling up" while tracking a medical issue is tonally off. *Suggestion: keep streaks, dial back the game mechanics.*

> [!] Items 3–6 are **product decisions** — discuss with Hojir before changing them. They're listed so you understand the product's tensions, not as immediate coding tasks.

Part B — The code

B1. Set up your computer (one-time)

You need a **Mac** (iOS development requires macOS). Install these: 1. **Xcode** — from the Mac App Store (large download). Open it once to accept the license, and install an iOS Simulator. 2. **Node.js 20+** — https://nodejs.org (or use `nvm`). Check: `node --version`. 3. **Homebrew** — https://brew.sh (a package installer for Mac). 4. **CocoaPods & Watchman:** `brew install cocoapods watchman` 5. **Git** — usually already installed. Check: `git --version`. 6. Ask Hojir to **add you as a collaborator** on the GitHub repo (so you can push code).

B2. Get the project running

```
# 1. Clone the repo (Hojir will give you access)
git clone https://github.com/jajrjavanmardi-debug/gutwell-app.git
cd gutwell-app

# 2. Install dependencies
npm install

# 3. Set up secrets — copy the example and fill in real values (ASK HOJIR for
them)
cp .env.example .env.local
#    Then edit .env.local. The client keys you need are:
#    EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_KEY, EXPO_PUBLIC_SENTRY_DSN
#    (PostHog / RevenueCat keys are optional — the app runs fine without them.)
#    NEVER put GROQ/USDA/GEMINI keys here — those are server-only secrets.

# 4. Build & run on the iOS simulator (first run takes ~10–15 min)
npx expo run:ios
```

> **Important:** GutWell uses "native modules" (notifications, in-app purchases, the widget), so it **cannot run in the Expo Go app** — you must use `npx expo run:ios`, which builds a real app. After the first build, you can use `npx expo start` for faster reloads.

Common first-run problems (and fixes): - `CocoaPods … Unicode Normalization` error -> run `export LANG=en_US.UTF-8` first, then re-run. - A Sentry "organization/project" error during build -> run `export SENTRY_DISABLE_AUTO_UPLOAD=true` first, then re-run. - "no such module" / a native error -> delete the `ios/` folder and run `npx expo run:ios` again (it regenerates it).

B3. Map of the codebase (where things live)

```
app/                 <- every screen (uses "Expo Router": folders = navigation)
  (auth)/            <- login, signup, forgot-password
  (onboarding)/      <- the 7-step welcome flow
  (tabs)/            <- the 5 main tabs: index(Home), checkin, food, progress,
profile
  photo-analysis.tsx <- the AI meal-photo wizard
  paywall.tsx, settings.tsx, weekly-digest.tsx, ... <- other screens
components/          <- reusable UI pieces (cards, charts, boxes, ErrorBoundary)
lib/                 <- the "brains" (no UI): scoring, streaks, correlations,
                        notifications, subscription, analytics, supabase client,
etc.
contexts/            <- AuthContext (who is logged in)
constants/theme.ts   <- colors, fonts, spacing (use these, don't hardcode)
supabase/
  migrations/        <- the database schema (SQL files, run in order)
  functions/         <- server code (the "edge function" that does AI safely)
widgets/             <- the iOS home-screen widget (Swift)
LAUNCH_OPERATIONS.md <- the launch checklist (store listing, keys, etc.)
```

B4. Key concepts (glossary)

- **Expo / React Native:** the framework. You write screens in `.tsx` files (TypeScript + React).
- **Expo Router:** navigation by folder structure — a file `app/settings.tsx` becomes a screen you can navigate to.
- **Supabase:** the backend — it stores users and their data (Postgres database), handles login, and runs server functions.
- **RLS (Row-Level Security):** database rules so users can only read/write *their own* rows. Already set up correctly — don't disable it.
- **Edge function:** server code (`supabase/functions/analyze-food`) that calls the AI (Gemini) using **secret keys that stay on the server**. The app never holds AI keys (that was a security bug we fixed).
- **RevenueCat:** the service that handles paid subscriptions. It "no-ops" (does nothing) until a key is set, so the app runs free without it.
- **PostHog / Sentry:** analytics and crash reporting. Optional; safe to run without.
- **EAS:** Expo's cloud build/submit service (used to make the App Store build).

B5. How to make a change safely (the workflow) — do this every time

```
git checkout main && git pull          # start from the latest
git checkout -b fix/short-description   # make a branch (never commit straight to
main)
# ... make your change in the code ...
npx tsc --noEmit                        # type-check: must show no NEW errors in
app/ or lib/
#   (errors inside supabase/functions are EXPECTED — that code runs on a
different
#    system called Deno; ignore those.)
git add <the files you changed>         # add ONLY your files (not node_modules,
not ios/)
git commit -m "fix: what you did"
git push -u origin fix/short-description
# then open a Pull Request on GitHub and ask Hojir to review before merging.
```

Rules: one small change per branch · never commit secrets (`.env.local`) or the `ios/` folder · always run `npx tsc --noEmit` before pushing · ask questions early.

Part C — What still needs to be done

Tasks are ordered **easiest -> hardest**. Start at the top to build confidence. Each task says **what**, **why**, **where**, and **how to check it worked**.

[EASY] Good first tasks (easy, safe, ~1 hour each)

1. **Fix the app name in the asset generator.** *What:* `scripts/generate-assets.mjs` still says `APP_DISPLAY_NAME = 'NutriFlow'` (the app's old name). *Why:* regenerating the icons would stamp the wrong name. *How:* change it to `'GutWell'`, run `npm run generate-assets`, look at `assets/images/`. *Check:* the icon wordmark says GutWell.

2. **Write a real README.** *What:* `README.md` is still the default Expo template. *Why:* it looks unfinished. *How:* replace it with a short intro (what GutWell is) + the "Get the project running" steps from Part B2.
3. **Remove dead code.** *What:* the `expo-device` package isn't used, and `app/reminders.tsx` (around line 18) imports `requestNotificationPermissions` but never uses it. *How:* `npm uninstall expo-device`, delete the unused import line. *Check:* `npx tsc --noEmit` is still clean.
4. **Add `supabase/config.toml`.** *What:* the Supabase project config file is missing. *Why:* it makes the database/edge-function reproducible. *How:* run `supabase init` (it will NOT overwrite the migrations). Commit the new `config.toml`.

[MEDIUM] Medium tasks (need some understanding, ~half a day each)

5. **Accessibility pass on login/signup/onboarding.** *What:* these screens have no accessibility labels, so blind users (VoiceOver) can't use them. *Why:* it's an App Store quality bar and the right thing to do. *Where:* `app/(auth)/*` and `app/(onboarding)/*`. *How:* on each text input add `accessibilityLabel="Email"` etc., and on each button add `accessibilityRole="button"` + a label. *Check:* turn on VoiceOver in the simulator (Settings -> Accessibility -> VoiceOver) and swipe through the screen.
6. **Make the "streak at risk" reminder actually fire.** *What:* `lib/notifications.ts` has a ready function `scheduleStreakAtRiskAlert(...)` that nothing calls. *Why:* it's meant to nudge users who haven't checked in. *Where:* call it from `app/(tabs)/checkin.tsx` (or app start) when today's check-in is missing, passing the current streak. *Check:* with notifications enabled on a real device/dev build, you get a reminder.
7. **Make notification taps open the right screen.** *What:* tapping a reminder just opens the app to wherever it was. *Where:* `app/_layout.tsx` — add `Notifications.addNotificationResponseReceivedListener(...)` and route based on the notification's `data.reminderType`. *Check:* tapping a check-in reminder opens the check-in screen.
8. **Fix premium unlocking after purchase.** *What:* after someone subscribes, the Progress/Weekly-digest screens don't unlock until you leave and come back. *Where:* `app/(tabs)/progress.tsx` and `app/weekly-digest.tsx` — the effect that calls `refreshPremiumStatus()` runs only once; switch it to re-check when the screen is focused (`useFocusEffect`). *Check:* premium content appears right after a (sandbox) purchase.
9. **Fix the legacy correlation time window.** *What:* one of the two correlation engines (`analyzeCorrelations` in `lib/correlations.ts`, ~line 236) can match a meal to a symptom across midnight; the newer engine was already fixed. *How:* apply the same "don't cross the end of the day" cap. *Check:* `npx tsc --noEmit` clean; numbers still look sensible.

[LARGER] Larger tasks (pair with Hojir / take your time)

10. **Add automated tests.** *What:* there are currently **zero tests**. *Where to start:* the pure-logic files `lib/scoring.ts`, `lib/streaks.ts`, `lib/correlations.ts`, `lib/offline-queue.ts` (no UI, easy to test). *How:* install Jest (`npx expo install jest jest-expo @types/jest`), add a `test` script, write small "given input -> expect output" tests. Great way to learn the code.

11. **Add Continuous Integration (CI).** *What:* a GitHub Actions workflow (`.github/workflows/ci.yml`) that runs `npx tsc --noEmit`, `npx expo lint`, and the tests on every Pull Request, so broken code can't be merged.
12. **Product improvements from Part A** (items 3–6: the fake "Analysing" screen, quiz personalization, the location feature, XP/levels). **These are product decisions — confirm with Hojir first.**

[OWNER-ONLY] Owner-only tasks (Hojir does these — they need company accounts & payment)
These gate the actual App Store launch and are **not** the intern's job, but you should know they exist (full detail in `LAUNCH_OPERATIONS.md`): - **Restore the paused Supabase project** (without this, the app can't load data) + upgrade to the paid plan so it stops pausing. - **Deploy the AI edge function** + set the server `GEMINI_API_KEY` / `USDA_API_KEY` secrets. - **Rotate the old leaked API keys** (Groq/USDA). - **Apple Developer enrollment** + fill the Apple IDs in `eas.json`. - **App Store Connect:** create the listing, upload screenshots, fill the privacy form (copy is ready in `LAUNCH_OPERATIONS.md`). - **Subscriptions** (only if launching paid): create the products in RevenueCat + App Store Connect.

Part D — How to proceed (suggested first two weeks)

Week 1 — get comfortable 1. Set up your Mac and get the app building (Part B1–B2). Ask Hojir for the `.env.local` values and repo access. 2. Click through every screen in the simulator. Read Part A so you understand the product. 3. Skim the code map (Part B3). Open `lib/scoring.ts` and `app/(tabs)/checkin.tsx` to see how UI and logic connect. 4. Do **good-first-tasks #1, #2, #3** — each as its own branch + Pull Request. This teaches you the workflow with low risk.

Week 2 — make real improvements 5. Do the **accessibility pass (#5)** — repetitive but safe and visible; great for learning the screens. 6. Do the **streak-alert wiring (#6)** and **notification taps (#7)**. 7. Start the **test suite (#10)** with Hojir's guidance — begin with `lib/scoring.ts`.

Ongoing - Keep Pull Requests **small** and ask for review. - Coordinate with Hojir on the [OWNER-ONLY] owner-only items — those unblock the launch. - When stuck for more than ~30 minutes, ask. That's normal and expected.

Appendix

What was already done (recent history)

PR	What
#21	Recovered the broken iOS widget so the app builds.
#22	Big cleanup: removed leaked API keys (moved AI to the server), renamed NutriFlow->GutWell, made notifications/analytics/subscriptions real, fixed core-logic bugs, deleted dead code.
#23	Fixed a crash, the onboarding "Enable Notifications" button (was fake), and added a top-level error screen.
#24	Added `LAUNCH_OPERATIONS.md` (the launch checklist).
#26	Made the gut score fair (honest logging no longer hurts it) + free correlation teaser before the paywall.

The code is **type-clean** (`tsc` passes) and the iOS build is verified working. Most remaining work is the owner-only launch steps plus the polish tasks above.

Feature inventory (what the app does today)

- **Onboarding:** welcome -> quiz -> "gut profile" -> notifications opt-in.
- **Auth:** login, signup, forgot-password.
- **Home:** gut score, trend sparkline, streak, daily tip, quick AI meal logging.
- **Daily check-in:** Bristol stool chart, symptoms, mood, water.
- **Food log:** AI photo analysis, manual entry, favourites, food<->symptom correlations.
- **Progress:** charts, calendar, mood trends, correlations (top one free, rest premium).
- **Profile:** stats, level/XP, achievements.
- **Extras:** weekly digest, reminders, settings (export + delete account), iOS widget, paywall.

Definition of done (v1 launch)

Must-have (owner): Supabase restored + paid plan · edge function deployed · keys rotated · Apple enrolled + `eas.json` filled · App Store listing + screenshots + privacy form · one TestFlight build. **Should-have (intern):** accessibility on auth/onboarding · a basic test suite + CI · the medium polish tasks. **Recommended:** launch **free first**, learn from real users (analytics + notifications are now real), add paid subscriptions in version 1.1.

Reference

- Launch checklist & store copy: **`LAUNCH_OPERATIONS.md`**
- This document also exists as **`GUTWELL_MASTER_BRIEF.docx`** and **`GUTWELL_MASTER_BRIEF.pdf`** (easier to share/read).
- Questions about the code? The `lib/` files have detailed comments explaining the "why."